

ΓΡΑΦΙΚΑ ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ
ΣΥΣΤΗΜΑΤΑ ΑΛΛΗΛΕΠΙΔΡΑΣΗΣ

1B ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΕΡΓΑΣΙΑ
ΓΙΑ ΤΟ ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ 2024-
2025

ΟΜΑΔΑ 5380-5226

ΙΩΑΝΝΗΣ ΦΙΛΛΗΣ, ΑΜ:5380

ΚΩΝΣΤΑΝΤΙΝΟΣ ΖΩΗΣ, ΑΜ:5226

ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

Νοέμβριος 2024

1 ΕΙΣΑΓΩΓΗ

Η παρούσα αναφορά συγγράφεται στα πλαίσια της δεύτερης εργαστηριακής άσκησης/προγραμματιστικής άσκησης (1B) του μαθήματος Γραφικά Υπολογιστών και Συστήματα Αλληλεπίδρασης από τους φοιτητές Γιάννη Φίλλη (ΑΜ: 5380) και Κωνσταντίνο Ζώη (ΑΜ:5226).

Σκοπός αυτής της άσκησης είναι να εξασκηθούμε στην χρήση των βασικών βιβλιοθηκών στοιχειωδών γραφικών της OpenGL. Ειδικότερα, στην συγκεκριμένη άσκηση έπρεπε να δημιουργήσουμε έναν 3D λαβύρινθο και να υλοποιήσουμε μία λειτουργία κάμερας.

2 ΕΡΩΤΗΜΑ 1: ΑΝΟΙΓΜΑ ΠΑΡΑΘΥΡΟΥ ΚΑΙ ΤΙΤΛΟΣ

Στο πρώτο ερώτημα, αυτό που καλούμαστε να κάνουμε, είναι αρχικά να δημιουργήσουμε το παράθυρο με διαστάσεις 950x950, να θέσουμε το background του παραθύρου σε χρώμα μαύρο, να θέσουμε τον τίτλο του παραθύρου **Άσκηση 1B-2024** και η εφαρμογή να τερματίζει με το πλήκτρο **SPACE**.

Έτσι λοιπόν, όπως φαίνεται και παρακάτω, πάμε στην εντολή **glfwCreateWindow()** και θέτουμε τις δύο παραμέτρους για το μέγεθος 950 και το όνομα στο επιθυμητό, ωστόσο θα πρέπει στην αρχή να βάλουμε το u8 ώστε να μπορεί να αναγνωρίσει τους ελληνικούς χαρακτήρες.

```
window = glfwCreateWindow(950, 950, u8"Άσκηση 1B - 2024", NULL, NULL);
```

Εικόνα 2.1: Εντολή δημιουργίας παραθύρου

Έπειτα για να αλλάξουμε το χρώμα του background σε μαύρο χρησιμοποιούμε την εντολή **glClearColor()** και με παραμέτρους 0.0f, 0.0f, 0.0f θέτουμε το χρώμα σε μαύρο (σε rgb).

```
//black background  
glClearColor(0.0f, 0.0f, 0.0f, 0.0f);
```

Εικόνα 2.2: Εντολή αλλαγής χρώματος σε μαύρο

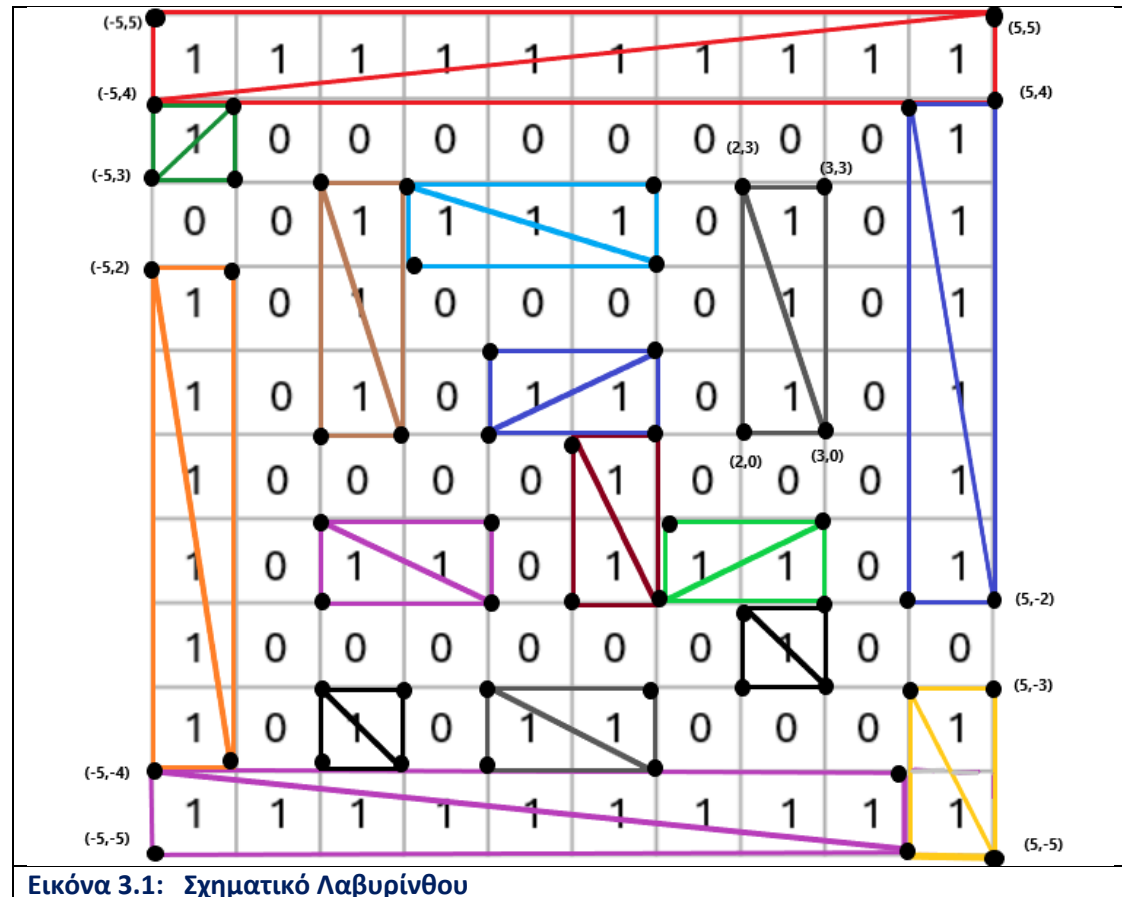
Για το τελευταίο ζητούμενο αυτού του ερωτήματος, θα βρούμε στον κώδικα το κύριο loop και θα τροποποιήσουμε την συνθήκη του, ώστε ο έλεγχος για το αν θα κλείσει το παράθυρο να γίνεται για το κουμπί SPACE.

```
} // Check if the ESC key was pressed or the window was closed  
while (glfwGetKey(window, GLFW_KEY_SPACE) != GLFW_PRESS &&  
       glfwWindowShouldClose(window) == 0);
```

Εικόνα 2.3: Εντολή αλλαγής χρώματος σε μαύρο

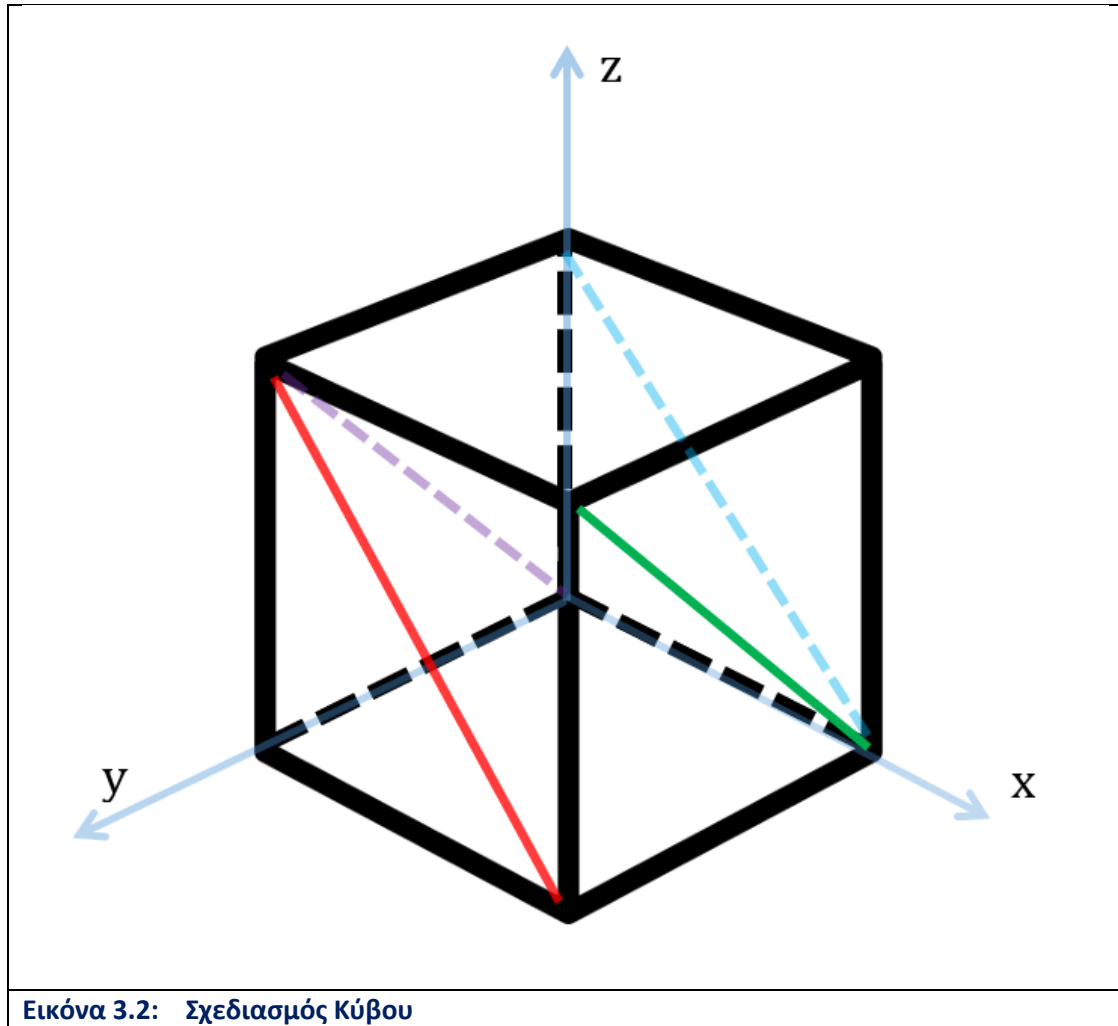
3 ΕΡΩΤΗΜΑ ΙΙ: ΛΑΒΥΡΙΝΘΟΣ

Για την σχεδίαση του λαβυρίνθου, χωρίσαμε τους τοίχους όπως και στην πρώτη άσκηση, με τα τρίγωνα που φαίνονται στην εικόνα 3.1. Θεωρούμε πως αυτό που βλέπουμε στην εικόνα είναι οι κάτω πλευρές των τοίχων, ότι ακουμπάνε δηλαδή στο επίπεδο xy . Τώρα πρέπει να μετατρέψουμε το κάθε ορθογώνιο ή τετράγωνο του λαβυρίνθου σε σχήμα τριών διαστάσεων.



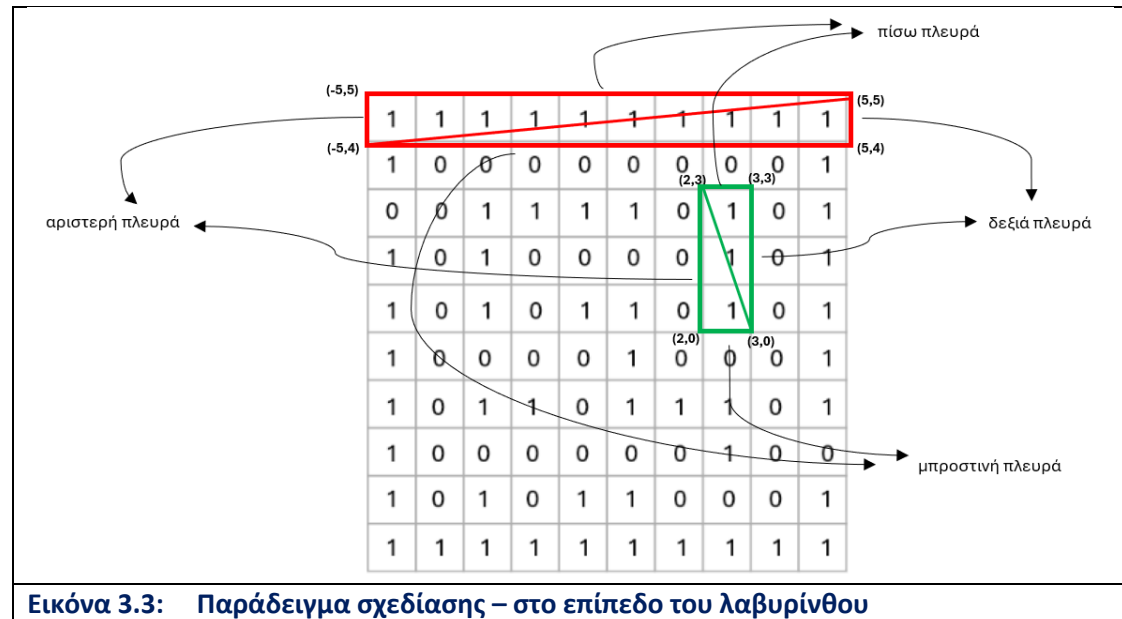
Εικόνα 3.1: Σχηματικό Λαβυρίνθου

Γνωρίζουμε ότι για να φτιάξουμε έναν κύβο σε 3 διαστάσεις στην OpenGL πρέπει να σχεδιάσουμε και τις 6 πλευρές του και να χωρίσουμε την κάθε πλευρά σε 2 τρίγωνα. Άρα χρειαζόμαστε 12 τρίγωνα για κάθε σχήμα που θα φτιάξουμε. Θεωρούμε λοιπόν έναν στοιχειώδες κύβο όπως αυτόν που φαίνεται στην εικόνα 3.2. Αυτό το σχήμα χρησιμοποιήσαμε σαν οδηγό για κάθε διαφορετικό block του λαβυρίνθου.

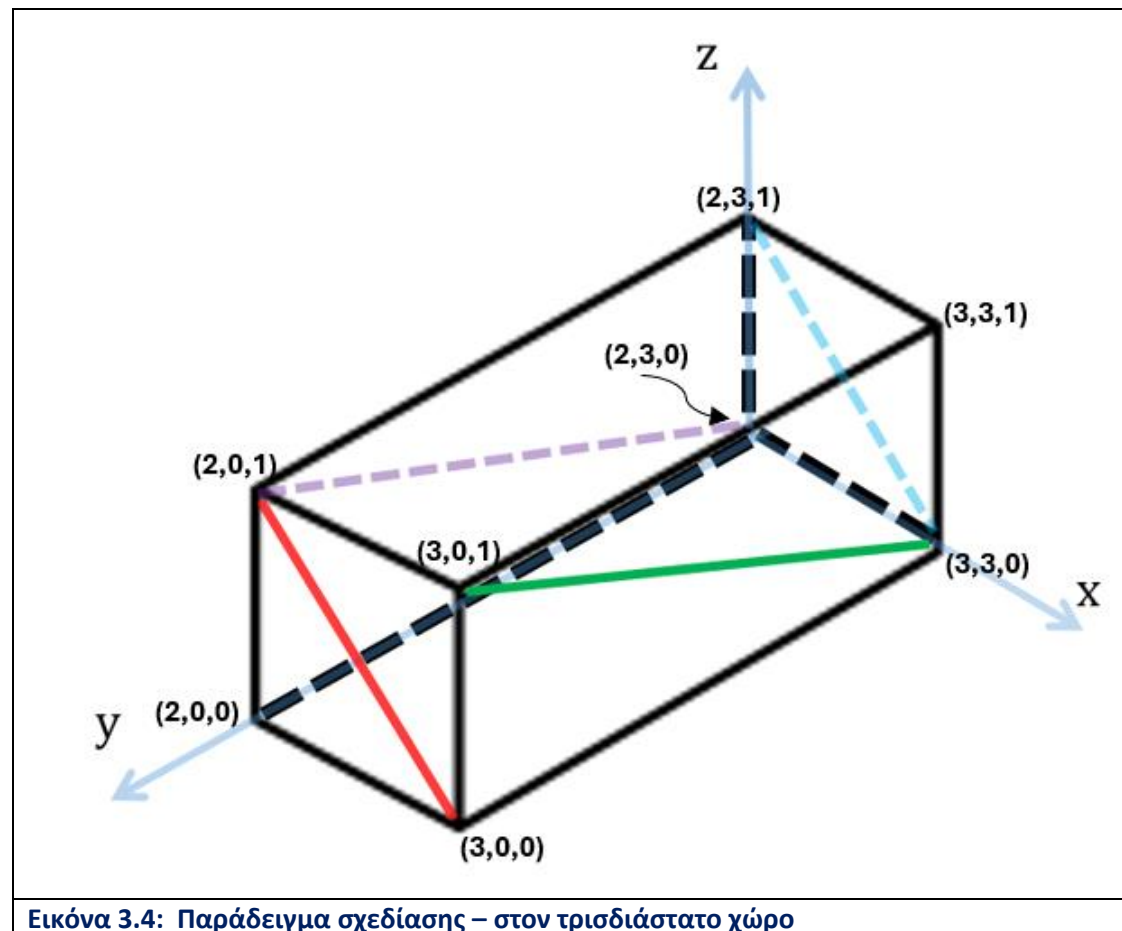
**Εικόνα 3.2: Σχεδιασμός Κύβου**

Σε αυτή την εικόνα βλέπουμε πως έχουμε χωρίσει τα τρίγωνα για την κάθε πλευρά του κύβου. Η μπροστινή πλευρά χωρίζεται από την κόκκινη τομή, η δεξιά πλευρά από την πράσινη, η αριστερή από την μωβ και η πίσω πλευρά από την γαλάζια. Η κάτω και η πάνω πλευρές χωρίζονται με τον ίδιο τρόπο και ανάλογα το block όπως στην εικόνα 3.1 όπου θεωρούμε ότι κοιτάμε τον λαβύρινθο από πάνω. Η πάνω πλευρά είναι ουσιαστικά ίδια με την κάτω απλά διαφέρει κατά 1 στα σημεία του άξονα z, βρίσκεται δηλαδή μία μονάδα ψηλότερα.

Όπως αναφέραμε, κάθε block του λαβυρίνθου χωρίζεται με τον ίδιο τρόπο που εξηγήσαμε παραπάνω. Ας δούμε ένα παράδειγμα:



Στην εικόνα 3.3 βλέπουμε πως αντιλαμβανόμαστε την κάθε πλευρά των σχημάτων στο επίπεδο του λαβυρίνθου. Ας δούμε τώρα συγκεκριμένα για το πράσινο ορθογώνιο, την μεταφορά του σε τρισδιάστατο χώρο και τον ορισμό των σημείων του. Οι τομές των πλευρών βασίζονται ξανά στον κύβο-οδηγό της εικόνας 3.2.



Τώρα ας δούμε τον ορισμό των vertices αυτού του σχήματος στην OpenGL μέσα στον πίνακα *maze_buffer*.

836	// (2,3) (3,3) (2,0) (3,0) block	858	//back
837	//bottom	859	2.0f, 3.0f, 0.0f,
838	2.0f, 3.0f, 0.0f,	860	2.0f, 3.0f, 1.0f,
839	2.0f, 0.0f, 0.0f,	861	3.0f, 3.0f, 0.0f,
840	3.0f, 0.0f, 0.0f,	862	3.0f, 3.0f, 0.0f,
841	2.0f, 3.0f, 0.0f,	863	3.0f, 3.0f, 1.0f,
842	3.0f, 3.0f, 0.0f,	864	2.0f, 3.0f, 1.0f,
843	3.0f, 0.0f, 0.0f,	865	//right
844	//up	866	3.0f, 0.0f, 0.0f,
845	2.0f, 3.0f, 1.0f,	867	3.0f, 0.0f, 1.0f,
846	2.0f, 0.0f, 1.0f,	868	3.0f, 3.0f, 0.0f,
847	3.0f, 0.0f, 1.0f,	869	3.0f, 3.0f, 0.0f,
848	2.0f, 3.0f, 1.0f,	870	3.0f, 3.0f, 1.0f,
849	3.0f, 3.0f, 1.0f,	871	3.0f, 0.0f, 1.0f,
850	3.0f, 0.0f, 1.0f,	872	//left
851	//front	873	2.0f, 0.0f, 0.0f,
852	2.0f, 0.0f, 0.0f,	874	2.0f, 0.0f, 1.0f,
853	2.0f, 0.0f, 1.0f,	875	2.0f, 3.0f, 0.0f,
854	3.0f, 0.0f, 0.0f,	876	2.0f, 3.0f, 0.0f,
855	3.0f, 0.0f, 0.0f,	877	2.0f, 3.0f, 1.0f,
856	3.0f, 0.0f, 1.0f,	878	2.0f, 0.0f, 1.0f,
857	2.0f, 0.0f, 1.0f,		

Εικόνα 3.5: Παράδειγμα σχεδίασης – αναπαράσταση στην OpenGL

Παρόμοια ορίζουμε και τα υπόλοιπα block του λαβυρίνθου αποθηκεύοντας τα vertices στον πίνακα *maze_buffer*. Για κάθε σχήμα έχουμε 8 μοναδικά σημεία, και 3 σημεία για κάθε τρίγωνο, η κάθε πλευρά έχει 2 τρίγωνα, άρα σχεδιάζουμε 36 σημεία. Παρατηρούμε επίσης ότι η πάνω πλευρά είναι ίδια με την κάτω αλλά με τη ζ συντεταγμένη ίση με 1, η πίσω πλευρά είναι ίδια με την μπροστινή αλλά με διαφορετικό y και η αριστερή πλευρά είναι ίδια με την δεξιά αλλά με διαφορετικό x.

Αφού ολοκληρώσουμε τον ορισμό των συντεταγμένων στον πίνακα, πρέπει να δημιουργήσουμε τα VBO για τον σχεδιασμό του λαβυρίνθου και για το χρώμα του.

1879	GLuint vertexbuffer;
1880	glGenBuffers(1, &vertexbuffer);
1881	glBindBuffer(GL_ARRAY_BUFFER, vertexbuffer);
1882	glBufferData(GL_ARRAY_BUFFER, sizeof(maze), maze, GL_STATIC_DRAW);
1883	
1884	GLuint colorbuffer;
1885	glGenBuffers(1, &colorbuffer);
1886	glBindBuffer(GL_ARRAY_BUFFER, colorbuffer);
1887	glBufferData(GL_ARRAY_BUFFER, sizeof(color), color, GL_STATIC_DRAW);
1888	

Εικόνα 3.6: Ορισμός των VBO

Στη συνέχεια χρειαζόμαστε έναν πίνακα για τα χρώματα ώστε να τον κάνουμε bind μετά με τον *colorbuffer*. Για το μπλε χρώμα η τρίτη μεταβλητή πρέπει να είναι ίση με 1 αφού δουλεύουμε σε τύπο RGB. Το μέγεθος του πίνακα *color* είναι ίσο με αυτό του λαβυρίνθου

επειδή πρέπει να περαστεί κάθε μπλοκ του **color** σε κάθε σχήμα του λαβυρίνθου. Επειδή έχουμε χωρίσει τον λαβύρινθο σε 16 σχήματα, 16 θα είναι και τα μπλοκ του χρώματος. Ένα παράδειγμα του πίνακα φαίνεται παρακάτω.

```
1189
1190 GLfloat a = 0.4f;
1191 static const GLfloat color[] = {
1192     0.0f, 0.0f, 1.0f, a,
1193     0.0f, 0.0f, 1.0f, a,
1194     0.0f, 0.0f, 1.0f, a,
1195     0.0f, 0.0f, 1.0f, a,
1196     0.0f, 0.0f, 1.0f, a,
1197     0.0f, 0.0f, 1.0f, a,
1198     0.0f, 0.0f, 1.0f, a,
1199     0.0f, 0.0f, 1.0f, a,
1200     0.0f, 0.0f, 1.0f, a,
1201     0.0f, 0.0f, 1.0f, a,
1202     0.0f, 0.0f, 1.0f, a,
1203     0.0f, 0.0f, 1.0f, a,
1204     0.0f, 0.0f, 1.0f, a,
1205     0.0f, 0.0f, 1.0f, a,
1206     0.0f, 0.0f, 1.0f, a,
1207     0.0f, 0.0f, 1.0f, a,
1208     0.0f, 0.0f, 1.0f, a,
1209     0.0f, 0.0f, 1.0f, a,
1210     0.0f, 0.0f, 1.0f, a,
1211     0.0f, 0.0f, 1.0f, a,
1212     0.0f, 0.0f, 1.0f, a,
1213     0.0f, 0.0f, 1.0f, a,
1214     0.0f, 0.0f, 1.0f, a,
1215     0.0f, 0.0f, 1.0f, a,
1216     0.0f, 0.0f, 1.0f, a,
1217     0.0f, 0.0f, 1.0f, a,
1218     0.0f, 0.0f, 1.0f, a,
1219     0.0f, 0.0f, 1.0f, a,
1220     0.0f, 0.0f, 1.0f, a,
1221     0.0f, 0.0f, 1.0f, a,
1222     0.0f, 0.0f, 1.0f, a,
1223     0.0f, 0.0f, 1.0f, a,
1224     0.0f, 0.0f, 1.0f, a,
1225     0.0f, 0.0f, 1.0f, a,
1226     0.0f, 0.0f, 1.0f, a,
1227     0.0f, 0.0f, 1.0f, a,
1228
```

Εικόνα 3.7: Το πρώτο μπλοκ του πίνακα color

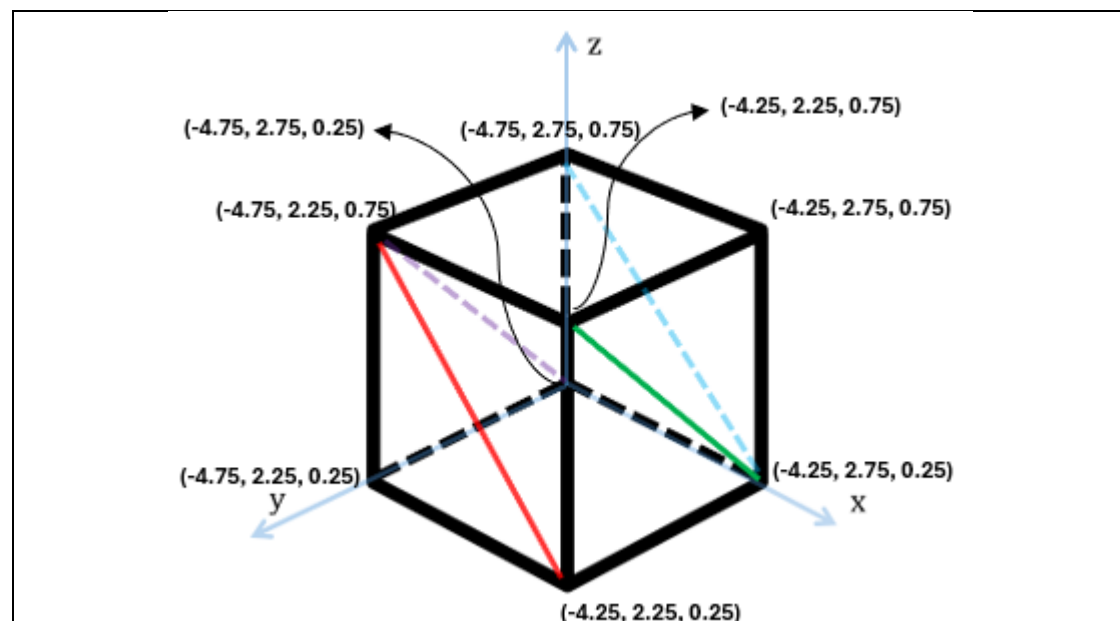
Τέλος, μας μένει να σχεδιάσουμε τον λαβύρινθο κάνοντας bind τα σημεία του (attribute 0) και χρωματίζοντας τον (attribute 1). Στην `glDrawArrays()` κάνουμε 16×36 καθώς όπως είπαμε ο λαβύρινθος αποτελείται από 16 σχήματα με 36 σημεία το καθένα.

```
1914 // 1st attribute buffer : vertices
1915 glEnableVertexAttribArray(0);
1916 glBindBuffer(GL_ARRAY_BUFFER, vertexbuffer);
1917 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, (void*)0);
1918
1919 // 2nd attribute buffer : colors
1920 glEnableVertexAttribArray(1);
1921 glBindBuffer(GL_ARRAY_BUFFER, colorbuffer);
1922 glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, 0, (void*)0);
1923
1924 // Draw triangles
1925 glDrawArrays(GL_TRIANGLES, 0, 16 * 36);
1926
1927
```

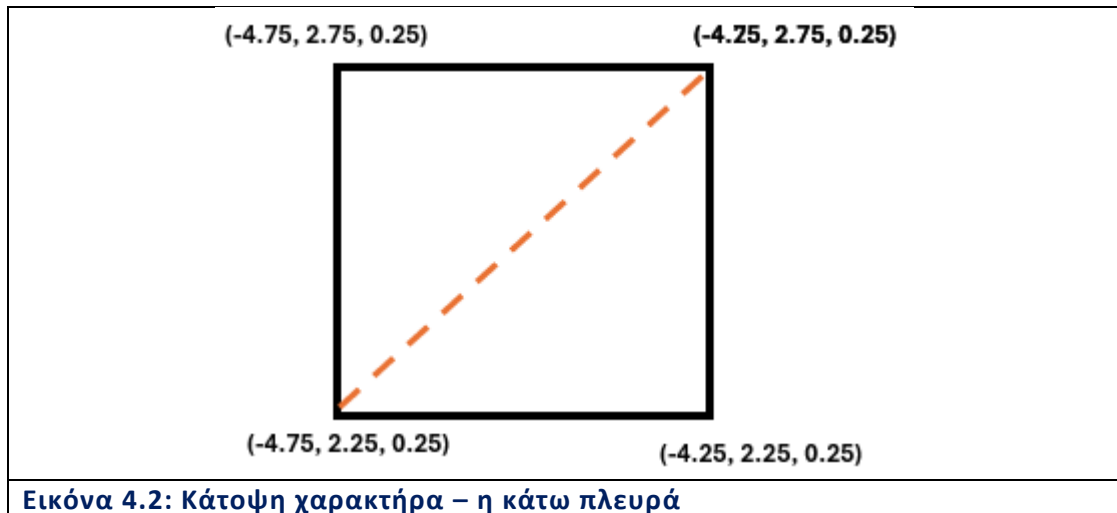
Εικόνα 3.8: Σχεδιασμός και χρωματισμός λαβυρίνθου

4 ΕΡΩΤΗΜΑ ΙΙΙ: ΧΑΡΑΚΤΗΡΑΣ

Ο χαρακτήρας σχεδιάζεται με παρόμοιο τρόπο όπως και κάθε σχήμα του λαβυρίνθου. Ο προσδιορισμός των συντεταγμένων του είναι ίδιος με την πρώτη άσκηση. Όμως επειδή τώρα δουλεύουμε σε 3 διαστάσεις και ο χαρακτήρας βρίσκεται στο κέντρο, ορίσαμε τις z συντεταγμένες του ως 0.75 για την πάνω πλευρά και 0.25 για την κάτω. Με αυτό τον τρόπο ο χαρακτήρας είναι ακριβώς στη μέση συγκριτικά με τους τοίχους και διατηρεί σχήμα κύβου. Πιο συγκεκριμένα, ο προσδιορισμός των πλευρών και ο διαχωρισμός των τριγώνων του είναι ίδιος με το υπόδειγμα που θεωρήσαμε αρχικά στην εικόνα 3.2.



Εικόνα 4.1: Προσδιορισμός τριγώνων και συντεταγμένων χαρακτήρα



Εικόνα 4.2: Κάτοψη χαρακτήρα – η κάτω πλευρά

Έτσι, ορίζουμε όλα τα σημεία στον πίνακα *character_buffer*.

```
1784
1785 static GLfloat character_buffer[] = {
1786     // (-4.75,2.25) (-4.25,2.25) (-4.75,2.75) (-4.25,2.75) block
1787     //bottom
1788     -4.75f, 2.75f, 0.25f, //xmin,ymax point
1789     -4.75f, 2.25f, 0.25f,
1790     -4.25f, 2.75f, 0.25f,
1791     -4.75f, 2.25f, 0.25f,
1792     -4.25f, 2.75f, 0.25f,
1793     -4.25f, 2.25f, 0.25f,
1794     //up
1795     -4.75f, 2.75f, 0.75f,
1796     -4.75f, 2.25f, 0.75f,
1797     -4.25f, 2.75f, 0.75f,
1798     -4.75f, 2.25f, 0.75f,
1799     -4.25f, 2.75f, 0.75f,
1800     -4.25f, 2.25f, 0.75f,
1801     //front
1802     -4.75f, 2.25f, 0.25f,
1803     -4.75f, 2.25f, 0.75f,
1804     -4.25f, 2.25f, 0.25f,
1805     -4.25f, 2.25f, 0.75f,
1806     -4.75f, 2.25f, 0.75f,
1807     -4.75f, 2.25f, 0.25f,
1808     //back
1809     -4.75f, 2.75f, 0.25f,
1810     -4.75f, 2.75f, 0.75f,
1811     -4.25f, 2.75f, 0.25f,
1812     -4.25f, 2.75f, 0.75f,
1813     -4.25f, 2.75f, 0.75f,
1814     -4.75f, 2.75f, 0.75f,
1815     //right
1816     -4.25f, 2.25f, 0.25f,
1817     -4.25f, 2.25f, 0.75f,
1818     -4.25f, 2.75f, 0.25f,
1819     -4.25f, 2.75f, 0.75f,
1820     -4.25f, 2.75f, 0.75f,
1821     -4.25f, 2.25f, 0.75f,
1822     //left
1823     -4.75f, 2.25f, 0.25f,
1824     -4.75f, 2.25f, 0.75f,
1825     -4.75f, 2.75f, 0.25f,
1826     -4.75f, 2.75f, 0.75f,
1827     -4.75f, 2.75f, 0.75f,
1828     -4.75f, 2.25f, 0.75f,
1829 };
1830
```

Εικόνα 4.3: Ο πίνακας *character_buffer*

Το χρώμα του χαρακτήρα θέλουμε να είναι κίτρινο, άρα θα ορίσουμε άλλον έναν πίνακα **color2** όπου η τιμή του κόκκινου και του πράσινου θα είναι ίση με 1 για να πάρουμε το κίτρινο χρώμα (εικόνα 4.4).

Φυσικά πρέπει να ορίσουμε και τα κατάλληλα VBO για τον χαρακτήρα (**characterbuffer**) και το νέο χρώμα (**colorbuffer2**) όπως φαίνεται στην εικόνα 4.5

```
1831
1832 static const GLfloat color2[] = {
1833     1.0f, 1.0f, 0.0f, a,
1834     1.0f, 1.0f, 0.0f, a,
1835     1.0f, 1.0f, 0.0f, a,
1836     1.0f, 1.0f, 0.0f, a,
1837     1.0f, 1.0f, 0.0f, a,
1838     1.0f, 1.0f, 0.0f, a,
1839     1.0f, 1.0f, 0.0f, a,
1840     1.0f, 1.0f, 0.0f, a,
1841     1.0f, 1.0f, 0.0f, a,
1842     1.0f, 1.0f, 0.0f, a,
1843     1.0f, 1.0f, 0.0f, a,
1844     1.0f, 1.0f, 0.0f, a,
1845     1.0f, 1.0f, 0.0f, a,
1846     1.0f, 1.0f, 0.0f, a,
1847     1.0f, 1.0f, 0.0f, a,
1848     1.0f, 1.0f, 0.0f, a,
1849     1.0f, 1.0f, 0.0f, a,
1850     1.0f, 1.0f, 0.0f, a,
1851     1.0f, 1.0f, 0.0f, a,
1852     1.0f, 1.0f, 0.0f, a,
1853     1.0f, 1.0f, 0.0f, a,
1854     1.0f, 1.0f, 0.0f, a,
1855     1.0f, 1.0f, 0.0f, a,
1856     1.0f, 1.0f, 0.0f, a,
1857     1.0f, 1.0f, 0.0f, a,
1858     1.0f, 1.0f, 0.0f, a,
1859     1.0f, 1.0f, 0.0f, a,
1860     1.0f, 1.0f, 0.0f, a,
1861     1.0f, 1.0f, 0.0f, a,
1862     1.0f, 1.0f, 0.0f, a,
1863     1.0f, 1.0f, 0.0f, a,
1864     1.0f, 1.0f, 0.0f, a,
1865     1.0f, 1.0f, 0.0f, a,
1866     1.0f, 1.0f, 0.0f, a,
1867     1.0f, 1.0f, 0.0f, a,
1868     1.0f, 1.0f, 0.0f, a
1869 };
1870
```

Εικόνα 4.4: Ο πίνακας color2

```
GLuint characterbuffer;  
glGenBuffers(1, &characterbuffer);  
  
GLuint colorbuffer2;  
glGenBuffers(1, &colorbuffer2);  
glBindBuffer(GL_ARRAY_BUFFER, colorbuffer2);  
glBufferData(GL_ARRAY_BUFFER, sizeof(color2), color2, GL_STATIC_DRAW);
```

Εικόνα 4.5: Ορισμός των VBO

Τέλος, για την τελική σχεδίαση του χαρακτήρα κάνουμε bind μέσα στο βρόγχο του πίνακά του ώστε να μπορεί να σχεδιάζεται δυναμικά και να αλλάζει η θέση του σε κάθε επανάληψη με το κατάλληλο input. Κάνουμε bind και το νέο χρώμα στο attribute 1 και είμαστε έτοιμοι να καλέσουμε την **glDrawArrays()** με όρισμα 36 αυτή τη φορά αφού έχουμε μόνο έναν κύβο.

```
1928  
1929 // character  
1930 glEnableVertexAttribArray(0);  
1931 glBindBuffer(GL_ARRAY_BUFFER, characterbuffer);  
1932 glBufferData(GL_ARRAY_BUFFER, sizeof(character_buffer), character_buffer, GL_DYNAMIC_DRAW);  
1933  
1934 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, (void*)0);  
1935  
1936 // color character  
1937 glEnableVertexAttribArray(1);  
1938 glBindBuffer(GL_ARRAY_BUFFER, colorbuffer2);  
1939 glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, 0, (void*)0);  
1940  
1941 //draw character  
1942 glDrawArrays(GL_TRIANGLES, 0, 36);  
1943
```

Εικόνα 4.6: Σχεδιασμός και χρωματισμός χαρακτήρα

5 ΕΡΩΤΗΜΑ IV: ΡΥΘΜΙΣΕΙΣ ΚΑΜΕΡΑΣ

Στο συγκεκριμένο ερώτημα, αυτό που καλούμαστε να κάνουμε είναι να ρυθμίσουμε κατάλληλα την κάμερά μας. Έτσι, θα χρησιμοποιήσουμε την εντολή **lookAt()**. Η πρώτη παράμετρος αφορά την αρχική θέση της κάμερας, οπότε εκεί θα βάλουμε ένα διάνυσμα με τιμές (0.0, 0.0, 20.0).

```
// Camera matrix  
ViewMatrix = glm::lookAt(  
    glm::vec3(0.0f, 0.0f, 20.0f),  
    glm::vec3(0.0f, 0.0f, 0.25f),  
    glm::vec3(0.0f, 1.0f, 0.0f)); // Head is up
```

Εικόνα 5.1: Εντολή lookAt και ρύθμιση των παραμέτρων της

Η δεύτερη παράμετρος αφορά το προς τα πού κοιτάει, οπότε θα θέσουμε το διάνυσμα (0.0, 0.0, 0.25) ενώ η τρίτη παράμετρος αφορά το ανιόν διάνυσμα, το οποίο το θέτουμε ίσο με (0.0, 1.0, 0.0).

Τον πίνακα που επιστρέφει θα τον θέτουμε ίσο με το **ViewMatrix**.

6 ΕΡΩΤΗΜΑ V: ΚΙΝΗΣΗ ΚΑΙ ΛΑΒΥΡΙΝΘΟΣ

Σχετικά με την κίνηση του χαρακτήρα και τη σύγκρουση σε τοίχους, η υλοποίηση είναι ίδια με την πρώτη άσκηση: Ορίζουμε την αναπαράσταση του χάρτη του λαβυρίνθου με την **createMap()**, με την **checkMove()** μέσω της **upscalePlayer()** ελέγχουμε αν η κίνηση είναι έγκυρη και αν ναι, την πραγματοποιούμε με την συνάρτηση **moveSquare()**. Το μόνο που αλλάζει είναι ότι θα πρέπει να μετακινήσουμε και τις 6 πλευρές αυτή τη φορά, επομένως στα for η επανάληψη θα έχει όριο 108 γιατί διατρέχουμε όλα τα νέα σημεία του κύβου (18 συντεταγμένες η κάθε πλευρά * 6 πλευρές). Για τον έλεγχο της σύγκρουσης πάλι παίρνουμε το πρώτο σημείο που ορίσαμε στον πίνακα και ξέρουμε ότι αυτό είναι το (x_{min}, y_{max}) . Αρκεί να γίνει ο έλεγχος μόνο με αυτό το σημείο ξανά καθώς οι πλευρές του κύβου δεν τον επηρεάζουν.

Η παρούσα άσκηση έχει την διαφορά ότι θέλουμε ο χαρακτήρας να μεταφέρεται στην έξοδο αν κινηθεί αριστερά από την είσοδο και να μεταφέρεται στην είσοδο αν κινηθεί δεξιά από την έξοδο. Για να το καταφέρουμε αυτό, υλοποιήσαμε τις δύο συναρτήσεις **isAtStart()** και **isAtEnd()** οι οποίες ελέγχουν αν το σημείο (x_{min}, y_{max}) βρίσκεται στη θέση της εκκίνησης ή του τερματισμού αντίστοιχα ελέγχοντας επακριβώς τις συντεταγμένες του.

```
213
214  bool isAtEnd(GLfloat* player) {
215      //check (xmin,ymax) point of exit block
216      if (player[0] == 4.25f && player[1] == -2.25f) {
217          return true;
218      }
219      else {
220          return false;
221      }
222  }
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

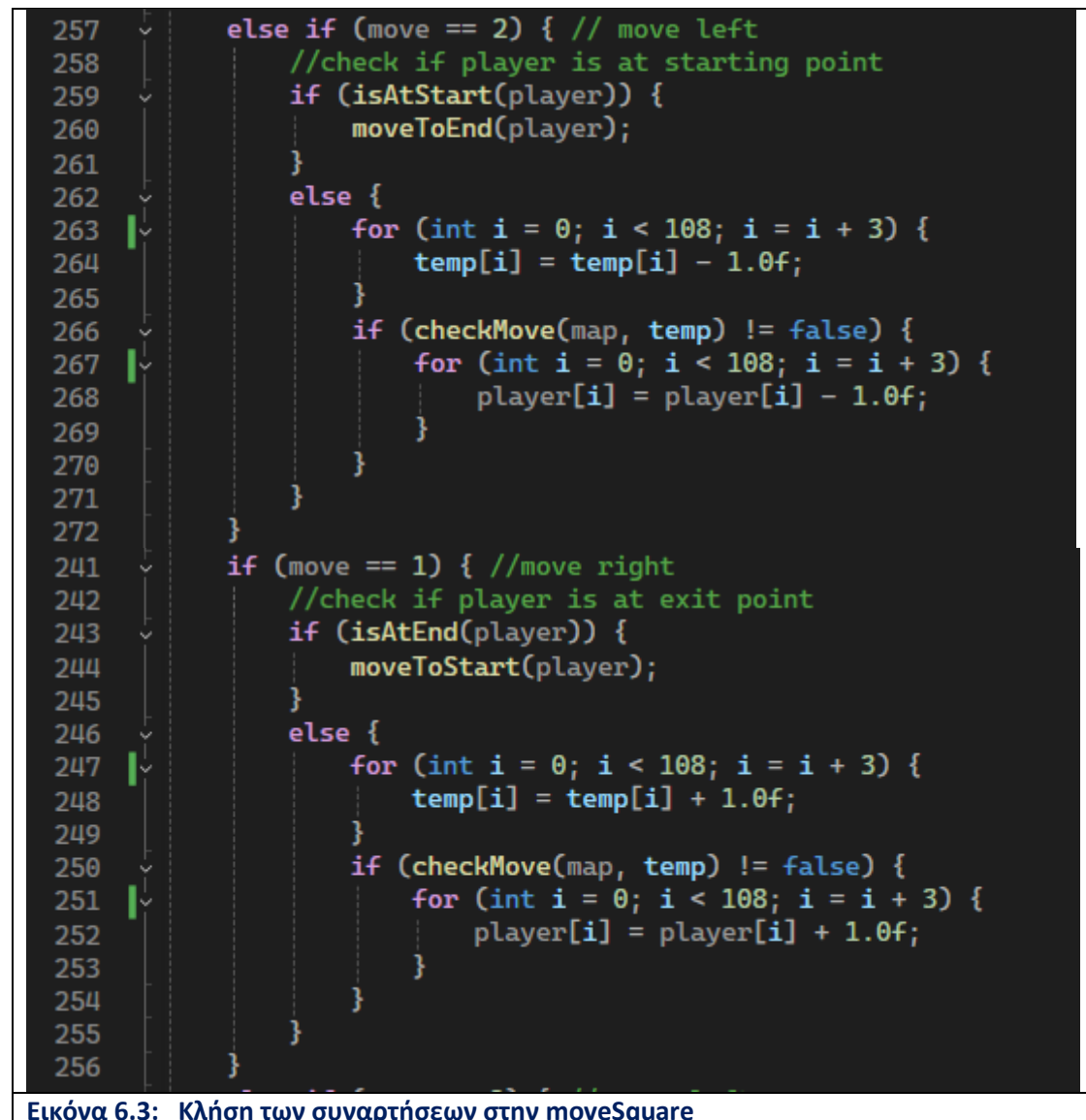
Εικόνα 6.1: Οι συναρτήσεις isAtStart και isAtEnd

Στη συνέχεια, οι συναρτήσεις **moveToStart()** και **moveToEnd()** μεταφέρουν τον χαρακτήρα στην αρχή και αντίστοιχα στο τέλος του λαβυρίνθου. Αλλάζουν όλα τα σημεία του για το x και το y ξεχωριστά. Όλα τα x θα διαφέρουν κατά μια συγκεκριμένη απόσταση και το y το ίδιο. Επομένως, αφού οι θέσεις προς και από την μεταφορά είναι συγκεκριμένες και τις γνωρίζουμε, αρκεί να προσθέσουμε ή να αφαιρέσουμε τη διαφορά τους κατά τις x συντεταγμένες και το ίδιο κατά τις y.

```
202
203 void moveToStart(GLfloat* player) {
204     //x
205     for (int i = 0; i < 108; i = i + 3) {
206         player[i] = player[i] - 9.0f;
207     }
208     //y
209     for (int i = 1; i < 108; i = i + 3) {
210         player[i] = player[i] + 5.0f;
211     }
212 }
213
224 void moveToEnd(GLfloat* player) {
225     //x
226     for (int i = 0; i < 108; i = i + 3) {
227         player[i] = player[i] + 9.0f;
228     }
229     //y
230     for (int i = 1; i < 108; i = i + 3) {
231         player[i] = player[i] - 5.0f;
232     }
233 }
234
```

Εικόνα 6.2: Οι συναρτήσεις `moveToStart` και `moveToEnd`

Οι συναρτήσεις αυτές καλούνται από την `moveSquare()` κατά την κίνηση του χαρακτήρα. Άρα βάζουμε ελέγχους if μέσα στην συνάρτηση `moveSquare()`. Στην κίνηση δεξιά ελέγχουμε αρχικά με την `isAtEnd()` αν ο χαρακτήρας βρίσκεται στο τέλος και αν ναι καλούμε την `moveToStart()` για την μεταφορά του στην εκκίνηση. Παρόμοια, στην κίνηση αριστερά ελέγχουμε αν ο χαρακτήρας βρίσκεται στην αρχή του λαβυρίνθου (`isAtStart()`) και αν ναι τον μεταφέρουμε στο τέλος (`moveToEnd()`).

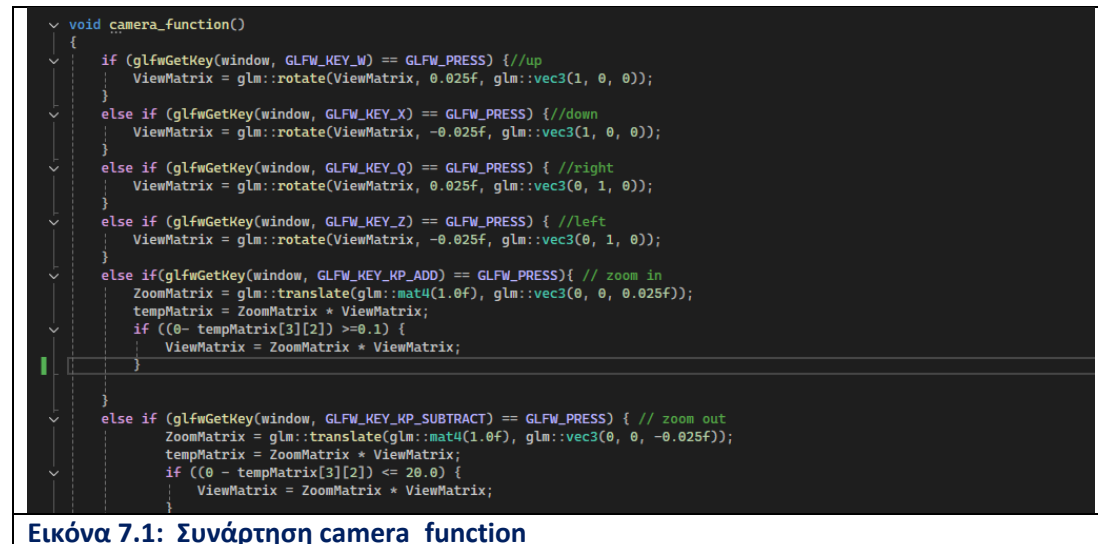


Εικόνα 6.3: Κλήση των συναρτήσεων στην `moveSquare`

7 ΕΡΩΤΗΜΑ VI: ΕΛΕΓΧΟΣ ΚΑΜΕΡΑΣ

Στο συγκεκριμένο ερώτημα θέλουμε να μετακινούμε την κάμερα, ανάλογα με το πάτημα των κουμπιών. Θέλουμε να μετακινείται γύρω από τους άξονες x και y αλλά να υλοποιήσουμε και μία λειτουργία zoom in/ zoom out.

Έτσι, θα υλοποιήσουμε την συνάρτηση `camera_function()`. Για την κάθε λειτουργία θα χρησιμοποιήσουμε ένα if statement. Αρχίζοντας με το rotation στον άξονα x, ελέγχουμε για το πάτημα του κουμπιού W και αν διαπιστώσουμε πως πατήθηκε, τότε χρησιμοποιούμε την ***rotate()*** με παραμέτρους τον πίνακα **ViewMatrix**, την τιμή 0.025 η οποία είναι το πόσα radians θα μετακινηθούμε και τέλος ένα διάνυσμα (1, 0, 0) το οποίο εκφράζει τον άξονα περιστροφής. Τον πίνακα που επιστρέφει τον θέτουμε ίσο με το **ViewMatrix**. Ομοίως, κάνουμε παρακάτω έλεγχο για πάτημα του κουμπιού X και καλούμε την `rotate` με ίδιες παραμέτρους εκτός από τα radians όπου τώρα είναι -0.025.



Στο τρίτο else if, κάνουμε έλεγχο για το πάτημα του κουμπιού Q και έπειτα καλούμε την rotate με ίδιες παραμέτρους αλλά τώρα το διάνυσμα που εκφράζει τον άξονα περιστροφής είναι (0,1,0) εφόσον θέλουμε περιστροφή στον άξονα y. Παρακάτω ελέγχουμε για το πάτημα του κουμπιού Z και καλούμε την rotate ξανά με τιμή για την μετακίνηση -0.025. Στα δύο τελευταία else if, κάνουμε ελέγχους για πάτημα των κουμπιών + και - του NumPad και έπειτα καλούμε την translate (αντί της rotate) με ορίσματα τον μοναδιαίο πίνακα και ένα διάνυσμα που εκφράζει την μετακίνηση την οποία θέλουμε να πραγματοποιήσουμε (για αυτό βλέπουμε τιμές 0 στους άξονες x και y και 0.025, -0.025 στον άξονα z, διότι ο τρόπος που υλοποιήσαμε την λειτουργία zoom in/zoom out είναι πως θα πραγματοποιείται μετακίνηση στον άξονα z).

Τον πίνακα που επιστρέφει η συνάρτηση translate τον θέτουμε ίσο με έναν πίνακα που ονομάζεται **ZoomMatrix** και είναι ουσιαστικά ένας προσωρινός βοηθητικός πίνακας που έχουμε ορίσει. Έπειτα, έχουμε έναν δεύτερο προσωρινό πίνακα που τον ονομάζουμε **tempMatrix** στον οποίο αποθηκεύουμε την τιμή που θα πάρει ο **ViewMatrix**. Κάνουμε έλεγχο ώστε να δούμε εάν η απόσταση από το αντικείμενό μας φτάσει να είναι πολύ μικρή (ή στην περίπτωση του zoom out, να είναι μεγάλη, δηλαδή η κάμερα να βρίσκεται στο αρχικό σημείο) τότε δεν κάνουμε ανάθεση στον **ViewMatrix** την τιμή. Τέλος, αν η απόσταση βρίσκεται μεταξύ αποδεκτών ορίων, και στις δύο περιπτώσεις θέτουμε το **ViewMatrix** ίσο με το γινόμενο του **ZoomMatrix** και της προηγούμενης τιμής του **ViewMatrix**.



Ο λόγος που χρησιμοποιούμε τις τιμές 0.025 και -0.025 για τους μετασχηματισμούς είναι επειδή έπειτα από διάφορες δοκιμές που κάναμε καταλήξαμε πως αυτή είναι μία αρκετά καλή τιμή η οποία προσφέρει μία πιο “smooth” μετακίνηση.

Ολοκληρώνοντας, καλούμε την **camera_function()** μέσα στον βρόγχο do-while, πριν κάνουμε τον υπολογισμό για το **MVP**.


```
// Clear the screen
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

// Use our shader
glUseProgram(programID);

camera_function();

ProjectionMatrix = getProjectionMatrix();
ViewMatrix = getViewMatrix();
ZoomMatrix = ViewMatrix;
glm::mat4 Model = glm::mat4(1.0f);

glm::mat4 MVP = ProjectionMatrix * ViewMatrix * Model;
```

Εικόνα 7.3: Κλήση της camera_function()

8 ΠΛΗΡΟΦΟΡΙΕΣ ΣΧΕΤΙΚΑ ΜΕ ΤΗΝ ΥΛΟΠΟΙΗΣΗ

Κατά την διάρκεια της άσκησης εργαστήκαμε με τα ακόλουθα:

Υπολογιστής 1

Λειτουργικό Σύστημα	Windows 10
Περιβάλλον/editor	Visual Studio 2022 (x64)

Υπολογιστής 2

Λειτουργικό Σύστημα	Windows 11
Περιβάλλον/editor	Visual Studio 2022 (x64)

Δεν υπάρχουν κάποιες ιδιαιτερότητες στον τρόπο εκτέλεσης ή χρήσης.

9 ΑΞΙΟΛΟΓΗΣΗ ΤΗΣ ΛΕΙΤΟΥΡΓΙΑΣ ΤΗΣ ΟΜΑΔΑΣ

Κατά την διάρκεια της εργασίας η συνεργασία μας ήταν άριστη. Πριν ξεκινήσουμε οποιοδήποτε ερώτημα, συζητάγαμε πως θα μπορούσε να υλοποιηθεί το καθένα, αφού αυτό είναι ουσιαστικά σχεδόν πάντα η κύρια δυσκολία. Αφού καταλήγαμε κάπου, ξεκινούσε η υλοποίηση (το προγραμματιστικό κομμάτι δηλαδή).

Το τελευταίο αυτό κομμάτι, προσπαθήσαμε όσο γίνεται να το ισομοιράσουμε. Η συγγραφή της αναφοράς έγινε με όμοιο τρόπο.

10 ΠΗΓΕΣ

Παρακάτω αναφέρουμε κάποιες από τις πηγές τις οποίες συμβουλευτήκαμε κατά την διάρκεια της άσκησης, πέρα από τις οδηγίες που δόθηκαν στο εργαστήριο.

1. https://www.glfw.org/docs/3.3/input_guide.html
2. <https://www.opengl-tutorial.org/beginners-tutorials/tutorial-3-matrices/>
3. <https://www.opengl-tutorial.org/beginners-tutorials/tutorial-4-a-colored-cube/>
4. <https://www.opengl-tutorial.org/beginners-tutorials/tutorial-6-keyboard-and-mouse/>